1. Show that for logistic regression with $p(y=1|\mathbf{x}) = \sigma[\mathbf{w}\cdot\mathbf{x}+b]$, $p(y=1|\mathbf{x})$ is constant within hyperplanes in the N-dim space perpendicular to $\mathbf{w}$.

why is this the p logistic regression

In logistic regression, the model is for a **binary** label $y \in \{0,1\}$. So we want a function that takes a real-valued "score" and turns it into a **valid probability** in $(0,1)$.

Why $p(y=1 \mid x) = \sigma(w \cdot x + b)$?

1. **Start with a linear score**

$$z = w^\top x + b \quad (\text{any real number})$$

2. **Convert score to probability**

   We need something that maps $\mathbb{R} \to (0,1)$. The standard choice is the **sigmoid**:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

   So we set

$$p(y=1 \mid x) = \sigma(w^\top x + b)$$

   and automatically $0 < p < 1$.

3. **Key interpretation: linear log-odds**

   Logistic regression isn't "randomly choosing" sigmoid — it's equivalent to assuming the **log-odds** are linear in x:

$$\log \frac{p(y=1 \mid x)}{1-p(y=1 \mid x)} = w^\top x + b$$

   If you solve this equation for p, you get exactly the sigmoid form:

$$p = \frac{1}{1+e^{-(w^\top x+b)}} = \sigma(w^\top x + b)$$

Why does this make probability constant on certain hyperplanes?

Because $p(y=1 \mid x)$ depends on x **only through** the single scalar $w^\top x + b$.
So if two points x_1, x_2 satisfy

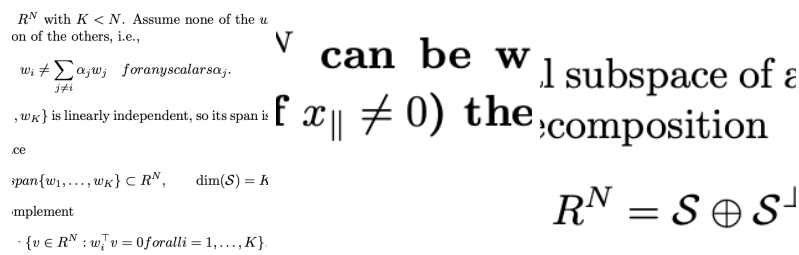
$$w^\top x_1 + b = w^\top x_2 + b,$$

then

$$p(y = 1 \mid x_1) = \sigma(w^\top x_1 + b) = \sigma(w^\top x_2 + b) = p(y = 1 \mid x_2).$$

The set of points with $w^\top x + b = c$ is a hyperplane whose normal vector is w, i.e., it's **perpendicular to** w.

If you want, I can also show the "move within the hyperplane" argument: if you move by a vector v with $w^\top v = 0$, the probability doesn't change.



why S is finite dim subspace, then R is equal to S and S org?

Great question — this is a **core linear algebra theorem**, and the sentence in the notes is using it a bit tersely.

Let me unpack it cleanly and rigorously.

What is being claimed?

You have

$$S = \operatorname{span}\{w_1, \ldots, w_K\} \subset \mathbb{R}^N,$$

with $K < N$, and

$$S^\perp = \{v \in \mathbb{R}^N : w_i^\top v = 0 \ \forall i\}.$$

The notes say:

> Since S is a finite-dimensional subspace of an inner product space,
>
> $$\mathbb{R}^N = S \oplus S^\perp.$$

You're asking: **why does "finite-dimensional subspace" imply this decomposition?**

The key theorem (Orthogonal Decomposition Theorem)

> **Theorem.**
> If V is a finite-dimensional inner product space and $S \subset V$ is any subspace, then
>
> $$V = S \oplus S^\perp.$$
>
> That is, every vector $x \in V$ can be written **uniquely** as
>
> $$x = x_\parallel + x_\perp, \quad x_\parallel \in S,\ x_\perp \in S^\perp.$$

Here:

- $V = \mathbb{R}^N$
- $S = \mathrm{span}\{w_1, \dots, w_K\}$
- $S^\perp$ is its orthogonal complement

Why does this hold? (Intuition first)

Think geometrically:

- S is a K-dimensional "flat" inside $\mathbb{R}^N$
- $S^\perp$ consists of all directions **orthogonal** to that flat
- Together, they cover **all directions** in the space
- No direction belongs to both except the zero vector

So any vector x can be split into:

- a component **inside** S
- a component **orthogonal** to S

This is exactly what projection does.

Why finite-dimensional matters

In finite dimensions:

- Every subspace is **closed**
- Orthogonal projections **exist and are unique**
- Dimension counting works cleanly

Specifically:

$$\dim(S) = K, \quad \dim(S^\perp) = N - K,$$

so

$$\dim(S) + \dim(S^\perp) = N = \dim(\mathbb{R}^N).$$

That's already a strong hint that

$$\mathbb{R}^N = S \oplus S^\perp.$$

Formal proof sketch (clean and exam-ready)

1. **Existence**
   Let $x \in \mathbb{R}^N$.
   Since S is finite-dimensional, there exists an orthogonal projection $x_\parallel = \mathrm{proj}_S(x)$.
   Define

   $$x_\perp := x - x_\parallel.$$

   Then for all i,

   $$w_i^\top x_\perp = w_i^\top (x - x_\parallel) = 0,$$

   so $x_\perp \in S^\perp$.
2. **Uniqueness**
   Suppose

   $$x = s_1 + t_1 = s_2 + t_2, \quad s_i \in S,\ t_i \in S^\perp.$$

   Then

   $$s_1 - s_2 = t_2 - t_1 \in S \cap S^\perp.$$

But

$$S \cap S^\perp = \{0\},$$

so $s_1 = s_2$ and $t_1 = t_2$.

Thus,

$$\mathbb{R}^N = S \oplus S^\perp.$$

Connecting back to your problem statement

This directly justifies the claim:

> Show that every $x \in \mathbb{R}^N$ can be written as
>
> $$x = x_\parallel + x_\perp, \quad w_i^\top x_\perp = 0 \ \forall i.$$

And the extra condition:

> "if $x_\parallel \neq 0$, there exists at least one i such that $w_i^\top x_\parallel \neq 0$"

follows because:

- If $x_\parallel \in S$ is nonzero
- It cannot be orthogonal to **all** basis vectors of S
- Otherwise it would lie in $S \cap S^\perp = \{0\}$

One-sentence takeaway (the intuition you want to remember)

> In finite-dimensional inner product spaces, **every subspace plus its orthogonal complement exactly fills the whole space**, and every vector splits uniquely into "parallel" and "orthogonal" parts.

If you want, next I can:

- Rewrite this in **very compact exam-style proof**, or
- Show how this decomposition is used later (e.g., logistic regression invariance along $S^\perp$).

2. Consider the case for which $y_i \in \{0,1\}$, and $x_i \in \mathbb{R}^d$, for $d > 1$. We consider the logistic regression model

$$p_\theta(Y = y_i | X = x_i) = [\sigma(z_i)]^{y_i}[1 - \sigma(z_i)]^{1-y_i}, \qquad z_i = w^T\psi(x_i) + b \tag{2}$$

Assume we are given training data $\mathcal{D} = \{(x_i, y_i)\}_{i=1,N}$. Again, $\psi(x)$ is a mapping of data x to a d'-dimensional feature space.

- Provide explicit expressions for the derivative of $\ell_\theta(\mathcal{D}) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(Y = y_i|X = x_i)$ wrt b and the gradient of $\ell_\theta(\mathcal{D})$ wrt $w \in \mathbb{R}^{d'}$.

is this cross entropy loss?

Yes — **exactly**.
What you're looking at *is* the **binary cross-entropy loss** (also called **log loss** or **negative log-likelihood for Bernoulli logistic regression**).

Let me show this cleanly and connect every piece.

Step 1: The model likelihood

You're given

$$p_\theta(Y = y_i \mid X = x_i) = [\sigma(z_i)]^{y_i}\,[1 - \sigma(z_i)]^{1-y_i}, \quad z_i = w^\top \psi(x_i) + b,$$

with $y_i \in \{0, 1\}$.

This is exactly the **Bernoulli distribution** with parameter

$$p_i := \sigma(z_i).$$

Step 2: Take the log-likelihood

$$\log p_\theta(Y = y_i \mid X = x_i) = y_i \log \sigma(z_i) + (1 - y_i)\log(1 - \sigma(z_i)).$$

Step 3: Negative average log-likelihood

The loss is defined as

$$\ell_\theta(\mathcal{D}) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(Y = y_i \mid X = x_i).$$

Plugging in:

$$\ell_\theta(\mathcal{D}) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \sigma(z_i) + (1-y_i)\log(1-\sigma(z_i))\Big].$$

Step 4: Why this is cross-entropy

For binary labels, the **cross-entropy** between:

- true distribution $y_i \in \{0,1\}$
- predicted probability $p_i = \sigma(z_i)$

is

$$\mathrm{CE}(y_i, p_i) = -\big[y_i \log p_i + (1-y_i)\log(1-p_i)\big].$$

So your loss is

$$\ell_\theta(\mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}\mathrm{CE}(y_i, \sigma(z_i)).$$

✔ Same expression
✔ Same meaning
✔ Same gradients

Step 5: Intuition (important for exams)

- Logistic regression assumes **Bernoulli likelihood**
- Maximizing likelihood ⇔ minimizing **cross-entropy**
- Cross-entropy measures how well predicted probabilities match true labels
- It heavily penalizes **confident but wrong** predictions

One-line exam answer

> Yes. This loss is exactly the **binary cross-entropy loss**, equivalently the **negative log-likelihood of a Bernoulli distribution** under the logistic regression model.

If you want, next I can:

- Derive the gradients wrt b and w step-by-step (very exam-friendly), or
- Show why the gradient simplifies to $(\sigma(z_i) - y_i)\psi(x_i)$.

what are things i need to know about cross-entropy

Great question — this is one of those topics that shows up **everywhere** (ML, DL, exams, interviews), and people often "use it" without really *knowing* it.

I'll give you a **compact but deep checklist**: what you must know, why it matters, and the formulas you should be able to write instantly.

Cross-Entropy: What You *Must* Know

1️⃣ Definition (memorize this)

For **true distribution** p and **predicted distribution** q:

$$\boxed{H(p, q) = -\sum_x p(x) \log q(x)}$$

👉 Cross-entropy measures how well q approximates p.

2️⃣ Binary cross-entropy (logistic regression case)

For $y \in \{0, 1\}$, prediction $p = \sigma(z)$:

$$\boxed{\mathcal{L}(y, p) = -\Big[y \log p + (1 - y) \log(1 - p)\Big]}$$

This is the **most common form** you'll see.

3️⃣ Multiclass cross-entropy (softmax)

For one-hot label y_k, softmax outputs p_k:

$$\boxed{\mathcal{L} = -\sum_{k=1}^{K} y_k \log p_k}$$

Since y is one-hot:

$$\mathcal{L} = -\log p_{\text{true class}}$$

4️⃣ Why cross-entropy appears everywhere

Key fact:

$$\boxed{\text{Cross-entropy loss} = \text{Negative log-likelihood}}$$

- Logistic regression → Bernoulli likelihood
- Softmax classifier → Categorical likelihood
- Minimizing CE ⇔ Maximum likelihood estimation (MLE)

👉 **This is the real reason** we use it.

5️⃣ Relation to KL divergence (very exam-relevant)

$$\boxed{H(p, q) = H(p) + \mathrm{KL}(p \| q)}$$

- $H(p)$: entropy of true distribution (constant)
- Minimizing cross-entropy ⇔ minimizing KL divergence
- Minimum achieved when $q = p$

This gives the **information-theoretic meaning**.

6️⃣ Gradient magic (VERY IMPORTANT)

Binary logistic regression:

$$z = w^\top x + b, \quad p = \sigma(z)$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial z} = p - y}$$

Then:

$$\frac{\partial \mathcal{L}}{\partial w} = (p - y)x, \quad \frac{\partial \mathcal{L}}{\partial b} = p - y$$

👉 This simplicity is **not accidental** — sigmoid + CE are a *matched pair*.

7️⃣ Why NOT mean-squared error for classification

If you use MSE with sigmoid:

- Slower convergence
- Vanishing gradients
- Poor probabilistic interpretation

Cross-entropy:

- Strong gradients when wrong
- Penalizes confident mistakes heavily
- Correct likelihood model

8️⃣ Numerical stability tricks (practical + interview)

Never compute:

$$\log(\sigma(z)) \text{ or } \log(1 - \sigma(z))$$

Instead use:

- `logsigmoid`
- `softmax + logsumexp`
- library CE implementations

Example:

$$\log \sigma(z) = -\log(1 + e^{-z})$$

9️⃣ Interpretation in words (be able to say this)

> Cross-entropy measures the expected number of bits needed to encode true labels using the model's predicted probability distribution.

or

> It penalizes predictions that assign low probability to the true label.

🔟 Common pitfalls (know these!)

❌ Thinking CE is just "a heuristic loss"
❌ Forgetting it's tied to likelihood
❌ Confusing entropy vs cross-entropy
❌ Mixing MSE with probabilistic classifiers
❌ Ignoring numerical stability

Ultra-short exam cheat sheet

- Definition: $H(p, q) = -\sum p \log q$
- Binary CE: $-[y \log p + (1-y)\log(1-p)]$
- Equals NLL of Bernoulli
- Minimizes KL divergence
- Gradient: $\sigma(z) - y$
- Sigmoid + CE = perfect match

If you want next, I can:

- Give **one clean page of derivations** (exam-style)
- Show **why sigmoid + CE cancels ugly terms**
- Compare **CE vs hinge loss vs MSE**
- Give **intuitive plots** of CE behavior

Just tell me what level you want.

what about the gradient? I don't get it

Totally fair — the gradient looks “magical” until you do it once cleanly.

I’ll derive the key result you need for logistic regression:

$$\frac{\partial \mathcal{L}}{\partial z} = \sigma(z) - y$$

and then show how that gives gradients for w and b.

Setup (one data point)

Let

- $z = w^\top x + b$
- $p = \sigma(z) = \frac{1}{1+e^{-z}}$
- $y \in \{0, 1\}$

Binary cross-entropy loss:

$$\mathcal{L}(y, p) = -\big[y \log p + (1-y)\log(1-p)\big]$$

Since $p = \sigma(z)$, the loss is a function of z: $\mathcal{L}(z)$.

Step 1: derivatives you need

(A) Sigmoid derivative

$$p = \sigma(z) \quad \Rightarrow \quad \frac{dp}{dz} = p(1-p)$$

(B) Loss derivative w.r.t. p

$$\mathcal{L} = -\big[y \log p + (1-y)\log(1-p)\big]$$

Differentiate:

$$\frac{d\mathcal{L}}{dp} = -\left(y \cdot \frac{1}{p} + (1-y) \cdot \frac{-1}{1-p}\right) = -\frac{y}{p} + \frac{1-y}{1-p}$$

Step 2: chain rule to get $d\mathcal{L}/dz$

$$\frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dp} \cdot \frac{dp}{dz}$$

Plug in:

$$\frac{d\mathcal{L}}{dz} = \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) \cdot p(1-p)$$

Distribute:

- First term: $-\frac{y}{p} \cdot p(1-p) = -y(1-p)$
- Second term: $\frac{1-y}{1-p} \cdot p(1-p) = (1-y)p$

So:

$$\frac{d\mathcal{L}}{dz} = -y(1-p) + (1-y)p$$

Expand:

$$-y + yp + p - yp = p - y$$

✅ Final:

$$\boxed{\frac{d\mathcal{L}}{dz} = p - y = \sigma(z) - y}$$

That's the "magic simplification".

Step 3: gradients for w and b

Now use:

$$z = w^\top x + b$$

w.r.t. w

$$\frac{\partial z}{\partial w} = x$$

So:

$$\boxed{\nabla_w \mathcal{L} = (\sigma(z) - y)\, x}$$

w.r.t. b

$$\frac{\partial z}{\partial b} = 1$$

So:

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = \sigma(z) - y}$$

For the whole dataset (average over N)

If $\ell = \frac{1}{N}\sum_i \mathcal{L}_i$, then:

$$\boxed{\nabla_w \ell = \frac{1}{N}\sum_{i=1}^{N} (\sigma(z_i) - y_i)\, x_i}$$

$$\boxed{\frac{\partial \ell}{\partial b} = \frac{1}{N}\sum_{i=1}^{N} (\sigma(z_i) - y_i)}$$

(If you have features $\psi(x_i)$, just replace x_i by $\psi(x_i)$.)

Intuition (why $p - y$ makes sense)

- If $y = 1$ but p is small, then $p - y$ is **very negative** → gradient pushes z **up** → p increases.
- If $y = 0$ but p is large, then $p - y$ is **positive** → pushes z **down** → p decreases.
- If $p \approx y$, gradient is near 0 → you're already good.

So the gradient is basically:

$$(\text{prediction error}) = (\text{predicted prob}) - (\text{label})$$

If you paste your exact notation from the screenshot ($\psi(x)$, $w \in \mathbb{R}^{d'}$), I can rewrite the final gradients in *exactly* that form.